2026 IEEE Ural-Siberian Conference on Biomedical Engineering, Radioelectronics and Information Technology (USBEREIT) | 979-8-3195-4177-2/26/$31.00 ©2026 IEEE | DOI: 10.1109/USBEREIT70063.2026.11581043

Evaluating a Multilayered Defense Against Prompt Injection in LLM-Augmented Educational Systems

Kirill Starukh
Department of Radio Electronics and Telecommunications
Ural Federal University
Voronezh, Russia
cyrillestaruh@yandex.ru

Anton Koshelev
Department of Artificial Intelligence
Ural Federal University
Yekaterinburg, Russia
Anton.Koshelev@urfu.ru

***Abstract*—Prompt injection attacks represent a persistent threat to production systems built on large language models (LLMs), particularly in educational environments where system integrity and student data protection are critical. This work investigates the effectiveness of a multilayered defensive architecture against prompt injection in an LLM-augmented educational bot deployed in a university setting. The proposed approach combines pattern-based input validation, prompt engineering via a sandwich technique, and LLM-based semantic post-filtering as sequential defense layers. We consider both direct user-crafted attacks and instruction-like content that can be embedded in external text, reflecting the broadened attack surface of modern LLM applications. The architecture is evaluated under two complementary regimes: synthetic prompts covering representative attack patterns and benign queries, and real-world interactions collected during a controlled in-vivo red-team exercise with student participants. The analysis reports overall blocking behavior together with per-layer contributions and identifies typical bypass mechanisms and operational failure modes observed in deployment. The findings suggest that a coordinated defense-in-depth strategy can distribute the protective burden across stages, improving robustness to paraphrase, obfuscation, and previously unseen variants. The paper contributes deployment-oriented evidence supporting architectural diversity as a practical principle for securing educational LLM systems.**

***Index Terms*—prompt injection, LLM security, multilayered defense, AI safety, educational technology**

I. Introduction

Large language models (LLMs) are increasingly integrated into user-facing applications, including educational tools. In the present study, the protected application is an adaptive digital teacher persona: a RAG-based tutoring agent integrated with the Moodle LMS API and designed to improve student progress through proactive interaction. However, such systems remain susceptible to prompt injection attacks—adversarial instructions embedded in user input that manipulate model behavior and compromise expected output correctness [1]–[3]. This threat arises because LLMs process system instructions and user content as a single natural-language sequence, without an inherent boundary between trusted and untrusted segments [4], [5]. As a result, malicious input can override earlier instructions in the context window, leading to unauthorized actions, disclosure of sensitive information, and unsafe domain-specific outputs [4], [6]. In educational platforms, such failures may also conflict with institutional policy. In this setting, prompt injection may affect not only general dialogue safety but also retrieval-grounded tutoring behavior, course-related guidance, and LMS-linked student support.

Modern LLM applications often rely on external tools and retrieval-augmented generation (RAG), which further expands the attack surface because retrieved web content, documents, or API outputs may contain adversarial instructions interpreted as executable commands [7]. Despite the proliferation of mitigation techniques, including input filtering, prompt-level constraint reinforcement, and model-based moderation, the robustness of integrated multilayer defenses remains insufficiently characterized in deployment-oriented settings [3], [5]. This motivates the following research question: how effectively can a multilayered architectural defense mitigate prompt injection in a production LLM system?

This paper makes three contributions. First, it formalizes and evaluates a practical three-layer defense stack combining regex-based input validation, reinforced system prompting, and LLM-based semantic post-filtering. Second, it assesses this architecture under two complementary regimes: a manually constructed synthetic benchmark and real interaction logs from a deployed university chatbot during a controlled red-team exercise. Third, it reports both overall and per-layer results, enabling analysis of cumulative protection, layer-specific contributions, and residual failure modes in an educational deployment context.

II. Related Work

Prompt injection has been established as a practical failure mode of instruction-following LLMs. Early work shows that adversarial prompts can induce goal hijacking and prompt leaking [4], while later studies extend this threat beyond direct user input to indirect injection in retrieved content and prompt-in-content attacks embedded in uploaded or pasted materials [2], [7]. Recent evaluation further indicates that even strongly aligned models remain only partially resilient, despite gains from stronger safety tuning [1]. At the same time, recent surveys note that the prompt-security literature remains fragmented across threat models, definitions, and evaluation procedures, which complicates direct comparison of proposed defenses [3], [5].

Authorized licensed use limited to: UNIVERSIDAD VERACRUZANA. Downloaded on October 07,2026 at 00:32:15 UTC from IEEE Xplore. Restrictions apply.

Relevant defenses fall into three broad groups: low-cost lexical pre-filters, prompt-level instruction reinforcement, and semantic model-based moderation at the prompt or output level [3], [5]. Lexical filters are fast but brittle under paraphrase and obfuscation, while prompt-level defenses can strengthen adherence to trusted instructions, including more explicit mechanisms such as instruction referencing [8]. Semantic moderation offers broader coverage but adds latency and depends on the evaluator model itself [3], [5]. Thus, prior work in the present source base mainly establishes attack feasibility [2], [4], [7], surveys the defense landscape [3], [5], or proposes specialized mechanisms [8], whereas deployment-oriented evaluation of a simple heterogeneous layered stack remains less emphasized. This paper addresses that practical gap by evaluating a three-layer defense in an educational deployment using both synthetic attacks and live interaction data.

III. Methods

A. Defense Architecture

The deployed defense was implemented as a sequential pipeline with three stages, shown in Fig. 1, where each incoming user message was processed in a fixed order. The deployed agent was configured for Russian- and English-language interaction only; requests in other languages were treated as unsupported and did not enter the normal tutoring workflow. A request was treated as blocked as soon as a blocking decision was made at any layer.

The pipeline consisted of the following layers:

1) Input Validation Layer (Regex-Based): A rule-based pre-filter matched incoming text against a fixed set of approximately 27 manually defined patterns associated with prompt injection and abusive inputs. The rule set covered role-switching attempts, delimiter abuse, XML-like tag manipulation, explicit stop/end markers, direct requests to reveal hidden instructions, Base64-obfuscated payloads, execution-oriented markers (e.g., `execute`, `run`, `eval`), roleplay markers, and extreme spam patterns consisting of symbol noise or repeated-character flooding. Rules were defined for English- and Russian-language attack formulations, which corresponded to the only two supported interaction languages of the deployed agent. Inputs in other languages were rejected as unsupported before normal tutoring interaction and were therefore outside the evaluated threat model of this deployment. If any regex rule matched, the request was blocked immediately and a template refusal message was returned.

2) System Prompt Layer (Sandwich Technique): Requests that passed the regex layer were forwarded to the main tutoring model under a defended prompt template. The user message was enclosed within explicit structural tags and accompanied by security instructions reinforcing the educational scope and refusal policy. A request was counted as blocked at this layer when the model explicitly refused to comply, indicating that the request was unethical, unsafe, or outside the permitted educational domain.

3) LLM-Based Semantic Post-Filter (LLM Censor): Outputs generated by the main tutoring model were additionally screened by an LLM-based semantic filter that evaluated whether the produced response violated policy or safety constraints. This layer served as a final safeguard for cases in which the main model generated a response despite an unsafe or manipulative input. If the post-filter flagged the generated response as violating policy, the response was suppressed and a template refusal message was returned instead.

B. Model Configuration

Both the tutor and the semantic post-filter used `qwen/qwen3-235b-a22b-2507` via OpenRouter (tag `2507`) with fixed settings: temperature $= 0.3$, top-p $= 1.0$, and maximum output length $= 3000$ tokens. Generation was therefore stochastic.

C. Experimental Protocol

For each incoming request x, the system applied the following decision procedure:

1) If the regex layer matched any blocking rule on x, the request was blocked immediately and no further processing was performed.

2) If x passed the regex layer, it was sent to the main tutoring model under the defended sandwich-prompt configuration. If the model explicitly refused to comply on ethical, safety, or domain-scope grounds, the request was counted as blocked at the system-prompt layer.

3) If the main model produced a substantive response, that response was passed to the LLM-based semantic post-filter. If the post-filter classified the generated output as violating policy or safety constraints, the response was suppressed and the request was treated as blocked.

4) Otherwise, the generated answer was returned to the user as an allowed response.

Accordingly, the final system decision followed an OR-style blocking rule: a request was blocked if any layer blocked it. Per-layer catch rates were computed conditionally, i.e., only over attacks that reached the corresponding layer.

D. Datasets

Synthetic Dataset: 266 manually constructed examples (141 attacks, 125 benign) created before live testing and used only for evaluation. Attacks covered direct injection, context manipulation, role switching, off-topic diversion, and prompt confusion, including obfuscated or multilingual variants.

Real-World Dataset: 1,082 messages were collected during a one-day controlled red-team exercise with 56 volunteer students. The dataset was independently labeled by the author and two student volunteers; disagreements were resolved by consensus. Messages received binary labels (block-worthy / allow-worthy) and intent labels (prompt injection, policy abuse, noise/spam, benign). Participation was voluntary, no personal data beyond usernames were collected, and no separate formal ethics review is reported.

Authorized licensed use limited to: UNIVERSIDAD VERACRUZANA. Downloaded on October 07,2026 at 00:32:15 UTC from IEEE Xplore. Restrictions apply.

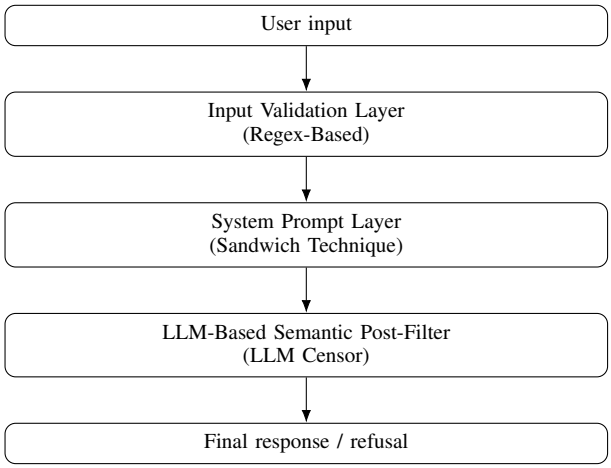


Fig. 1. Sequential defense pipeline applied to each user input before final response delivery.

TABLE I
OVERALL PERFORMANCE METRICS ON SYNTHETIC ATTACK DATASET.

Metric	Value
Attack Recall	97.87%
False Positive Rate	6.40%
Balanced Accuracy	95.74%
Multi-Turn Payload Recall	100.0%
True Positives / False Negatives	138 / 3

E. Metrics

Evaluation metrics: Attack Recall: proportion of attacks correctly blocked. False Positive Rate: proportion of benign queries incorrectly blocked. Balanced Accuracy: mean of true positive and true negative rates. Per-Layer Catch Rate: attacks blocked by each layer among those reaching it.

$$\mathrm{TPR} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}} \qquad \mathrm{TNR} = \frac{\mathrm{TN}}{\mathrm{TN} + \mathrm{FP}}$$

$$BA = \frac{TPR + TNR}{2}$$

$$= \frac{1}{2}\left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP}\right)$$

$$\mathrm{CatchRate}(L) = \frac{\mathrm{Blocked}(L)}{\mathrm{Reached}(L)}$$

IV. RESULTS AND DISCUSSION

A. Synthetic Data Performance

Evaluation of 266 synthetic examples (141 attacks, 125 benign) yielded the results summarized in Table I.

The system successfully blocked 138 of 141 attacks (97.87% recall), with only three false negatives. Eight benign queries were incorrectly blocked (FPR = 6.4%). Multi-turn payload-splitting attacks achieved 100% recall.

Per-layer decomposition is presented in Table II.

TABLE II
PER-LAYER ATTACK INTERCEPTION ON SYNTHETIC DATA.

Defense Layer	Attacks Reaching	Attacks Blocked	Catch Rate
Regex	141	32	22.70%
System Prompt	109	101	92.66%
LLM Censor	8	5	62.50%
Cumulative	141	138	97.87%

TABLE III
OVERALL PERFORMANCE METRICS ON REAL-WORLD INTERACTION DATASET.

Metric	Value
Overall Block-Worthy Recall	83.41%
False Positive Rate	5.42%
Balanced Accuracy	88.99%
True Positives / False Negatives	764 / 152
Allow-Worthy Correctly Permitted	157 / 166 (94.58%)

TABLE IV
PERFORMANCE BY ATTACK INTENT CATEGORY.

Type	N	TP	FN	Recall
Prompt Injection	482	410	72	85.06%
Policy Violation	232	211	21	90.94%
Noise/Spam	117	93	24	79.48%

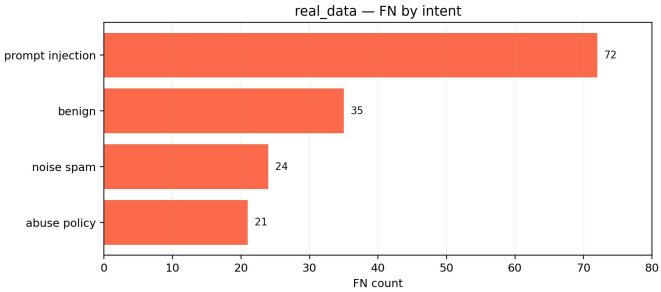


Fig. 2. False negatives by attack intent category.

B. Real-World User Interaction Data

Overall real-world performance is shown in Table III.

The system achieved 764 true positives with 152 false negatives and 9 false positives. The 5.42% false positive rate corresponds to approximately 54 benign queries blocked per 1,000 benign queries, representing a moderate trade-off in high-safety deployment contexts.

Performance stratified by intent category is detailed in Table IV.

Differential performance across attack types reflects varying sophistication: policy violations achieved highest recall (90.94%), while noise/spam attacks proved most difficult to detect (79.48%), suggesting that spam-like payloads closely resemble legitimate queries.

C. False Negative Breakdown and Safety Implications

As shown in Fig. 2, the distribution of false negatives across intent categories reveals critical safety considerations. Prompt injection attacks account for 72 undetected cases (47.4% of all false negatives), representing the largest vulnerability surface.

Authorized licensed use limited to: UNIVERSIDAD VERACRUZANA. Downloaded on October 07,2026 at 00:32:15 UTC from IEEE Xplore. Restrictions apply.

However, qualitative review of the annotated outputs suggested that some successful injections did not translate into overtly harmful responses. This observation is based on manual output annotation and is reported as a qualitative finding rather than as a separate benchmark metric. Policy violations constitute only 21 undetected cases (13.8% of FN) but represent the highest risk category. Abuse of taboo topics—politically incorrect statements, weapon manufacturing guidance, or similar harmful content—carries the greatest liability for developers. Qualitative inspection of the annotated outputs further suggested that the reinforced system prompt functioned as a secondary containment layer in some such cases, although this effect was not evaluated as a standalone quantitative metric.

Noise/spam false negatives (24 cases, 15.8% of FN) pose minimal risk. Such messages are typically content-void, and their primary threat concerns system abuse rather than prompt injection per se—a separate infrastructure concern beyond this study's scope. False positives (9 cases, 5.4% FPR) consisted of out-of-domain but benign queries. While ideally the system should restrict responses to educational topics, these false rejections cause no harm if the model answers; the greater priority is preventing harmful outputs. Key insight: The architectural defense and system prompt together create a defense-in-depth strategy. While detection remains imperfect (83.41% real-world recall), downstream semantic guardrails mitigate the most dangerous failure modes, ensuring that undetected injections do not translate into harmful model behavior. The results support the working assumption that a layered defense architecture can provide robust prompt-injection mitigation in this deployment setting. The multilayered architecture achieved 97.87% recall on synthetic data and 83.41% on real-world interactions. In contrast, prior studies have demonstrated high attack success rates against unprotected or single-mechanism systems. Perez and Ribeiro [4] report that language models frequently comply with adversarial prompt instructions, while Greshake et al. [7] demonstrate successful indirect prompt injection against real-world LLM-integrated applications. These findings indicate that single-layer or uncoordinated defenses remain highly vulnerable under realistic attack conditions. The present results provide empirical evidence that architectural composition can deliver robust prompt-injection mitigation in a deployment-oriented educational setting.

D. Discussion and Comparison with Prior Work

The results indicate that the proposed multilayer defense provides robust prompt-injection mitigation under the evaluated conditions. Prior work has focused mainly on attack construction or single-layer defenses: Perez and Ribeiro [4] analyze injection techniques and model susceptibility without architectural protection, while Greshake et al. [7] demonstrate the severity of indirect prompt injection in retrieval-augmented systems. Recent surveys further note that rule-based filtering, prompt engineering, and LLM-based moderation are often studied in isolation, while integrated multilayer evaluation remains limited [3], [5].

In contrast, the present work evaluates these mechanisms jointly within one deployed pipeline and reports both cumulative and per-layer performance. The system achieves 97.87% recall on synthetic attacks and 83.41% recall on real-world interaction data, including mixed attack categories, multi-turn payload splitting, and live deployment conditions. Beyond detection accuracy, the results also show a defense-in-depth effect: even when some attacks bypass earlier checks, reinforced system prompts and downstream semantic filtering can still reduce the risk of harmful outputs. This distinguishes the proposed architecture from single-layer approaches, where false negatives more directly translate into unsafe behavior.

A key limitation is that the real-world dataset was collected during a controlled red-team exercise rather than routine day-to-day student usage. Although the interactions came from a deployed university chatbot, the traffic is therefore more representative of adversarial testing than ordinary production behavior, and the observed recall and false-positive trade-offs should be interpreted accordingly.

V. Conclusion

A three-layer defense combining regex filtering, reinforced system prompting, and LLM-based semantic post-filtering improved robustness against prompt injection in this educational deployment. The system achieved 97.87% recall on synthetic data and 83.41% on live red-team interactions, indicating that no single layer carried the full defensive burden. These results support the practical value of heterogeneous defense-in-depth for production LLM systems.

References

[1] D. Ganiuly and A. Smaiyl, "Prompt injection as an emerging threat: Evaluating the resilience of large language models," arXiv:2511.01634, Nov. 2025, doi: 10.48550/arXiv.2511.01634.

[2] Z. Lian *et al.*, "Prompt-in-content attacks: Exploiting uploaded inputs to hijack LLM behavior," arXiv:2508.19287, 2025.

[3] H. Hong, S. Feng, N. Naderloui, S. Yan, J. Zhang, B. Liu, A. Arastehfard, H. Huang, and Y. Hong, "SoK: Taxonomy and evaluation of prompt security in large language models," arXiv:2510.15476, Oct. 2025, doi: 10.48550/arXiv.2510.15476.

[4] F. Perez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," arXiv:2211.09527, Nov. 2022, doi: 10.48550/arXiv.2211.09527.

[5] S. Gulyamov, S. Gulyamov, A. Rodionov, R. Khursanov, K. Mekhmonov, D. Babaev, and A. Rakhimjonov, "Prompt injection attacks in large language models and AI agent systems: A comprehensive review of vulnerabilities, attack vectors, and defense mechanisms," *Information*, vol. 17, no. 1, p. 54, 2026, doi: 10.3390/info17010054.

[6] R. W. Lee, T. J. Jun, J. Lee, S. I. Cho, H. J. Park, and J. Suh, "Vulnerability of large language models to prompt injection when providing medical advice," *JAMA Netw. Open*, vol. 8, no. 12, p. e2549963, Dec. 2025, doi: 10.1001/jamanetworkopen.2025.49963.

[7] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," arXiv:2302.12173, Feb. 2023, doi: 10.48550/arXiv.2302.12173.

[8] Y. Chen, H. Li, Y. Sui, Y. Liu, Y. He, Y. Song, and B. Hooi, "Robustness via referencing: Defending against prompt injection attacks by referencing the executed instruction," arXiv:2504.20472, Apr. 2025, doi: 10.48550/arXiv.2504.20472.

Authorized licensed use limited to: UNIVERSIDAD VERACRUZANA. Downloaded on October 07,2026 at 00:32:15 UTC from IEEE Xplore. Restrictions apply.